

Aula 8 Parte 1

Laboratório Comparativo de Classificação: Cinco Algoritmos, um Problema Real

Objetivos desta sessão

- Comparar **k-NN**, **regressão logística**, **árvore de decisão**, **Naive Bayes** e **SVM** no mesmo problema real (solicitações de empréstimo), com o mesmo pré-processamento, o mesmo treino e o mesmo teste.
- Usar **Pipeline** e **ColumnTransformer** para que o pré-processamento faça parte do modelo avaliado.
- Estabelecer um **baseline** como piso de qualquer comparação.
- Ler os resultados além da acurácia: **matriz de confusão**, **precisão**, **recall** e **F1**.
- Identificar quais modelos dependem da **escala** dos atributos e quais não dependem.
- Perceber que uma única divisão treino/teste é uma estimativa **ruidosa** para declarar "o melhor modelo".

1. O problema e a regra do jogo

Na aula sobre classificação conhecemos cinco famílias de algoritmos e vimos como cada uma desenha sua fronteira de decisão. Naquele momento usamos um conjunto sintético (duas "luas"), escolhido para deixar as fronteiras visíveis. Hoje o desafio é outro: colocar as mesmas famílias para competir num **problema real e barulhento** — o conjunto de solicitações de empréstimo que já preparamos nas aulas de tratamento de dados.

Para uma comparação ser justa, **apenas o algoritmo pode mudar**. Tudo o mais é mantido igual para os cinco competidores:

- os **mesmos dados** e a **mesma divisão** treino/teste (30% para teste, **random_state=42**, estratificada);
- o **mesmo pré-processamento** (imputação, padronização e one-hot), aprendido só com o treino;
- as **mesmas métricas**, calculadas sobre o mesmo conjunto de teste.

Se cada modelo recebesse um tratamento diferente, não saberíamos dizer se a diferença de desempenho vem do algoritmo ou de como os dados foram preparados.

1.1. Os dados

São 450 solicitações de empréstimo. O objetivo é prever **aprovado** (1 = aprovado, 0 = negado) a partir de cinco atributos:

Atributo	Tipo	Observação
idade	numérico	cerca de 9% de valores ausentes
renda	numérico	cerca de 9% ausentes; poucos outliers muito altos
tempo_emprego	numérico	sem valores ausentes
score_credito	numérico	escala de 300 a 850; sem valores ausentes
cidade	categórico nominal	cerca de 7% ausentes; SP, RJ, MG ou Outra

As duas classes são quase equilibradas (52% negados, 48% aprovados). Isso torna a acurácia uma métrica razoável **neste** problema — mas veremos que ela não conta a história toda.

2. Antes de comparar: o baseline

O **baseline** é um modelo que não aprende nada: ele sempre prevê a classe mais comum do treino (em `scikit-learn`, `DummyClassifier(strategy="most_frequent")`). Aqui ele acerta cerca de 52% — é o que se consegue "chutando" sempre "negado".

O baseline é o **piso** de qualquer comparação: um modelo que não o supera com folga não aprendeu nada útil. Ele também protege contra um erro comum, o de comemorar uma acurácia que parece alta sem saber quanto valia o chute.

Na divisão treino/teste usamos `stratify=y`, que mantém a mesma proporção de aprovados nos dois conjuntos. Sem isso, o sorteio poderia entregar um teste torto (com muito mais aprovados que o treino, por exemplo) e distorcer todas as métricas.

3. O pré-processamento dentro de um Pipeline

Os dados brutos não podem ir direto para os algoritmos: há valores ausentes e uma coluna de texto. Cada tipo de coluna recebe um tratamento próprio, reunido em um `ColumnTransformer`:

```
preprocessador = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                     ("esc", StandardScaler())]),
     ["idade", "renda", "tempo_emprego", "score_credito"]),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                     ("oh", OneHotEncoder(handle_unknown="ignore"))]),
     ["cidade"]),
])

modelo_completo = Pipeline([("prep", preprocessador), ("modelo", KNeighborsClassifier(5))])
modelo_completo.fit(X_treino, y_treino)      # aprende mediana, média, desvio e categorias SO no
treino
y_prev = modelo_completo.predict(X_teste)    # aplica no teste o que aprendeu no treino
```

A vantagem de colocar tudo dentro do `Pipeline` é que a mediana, a média, o desvio-padrão e a lista de categorias são calculados **somente com o treino** e depois apenas *aplicados* ao teste. Se o teste participasse desses cálculos, informação de dados que deveriam ser "nunca vistos" vazaria para o modelo. Esse cuidado — chamado de **vazamento de dados** — é assunto central da próxima sessão; hoje basta adotar o hábito de sempre usar o pipeline.

4. Os cinco competidores

A tabela retoma a ideia central de cada família e adiciona duas colunas que importam para a comparação de hoje: o tipo de fronteira e a sensibilidade à escala dos atributos.

Algoritmo	Ideia central	Fronteira típica	Sensível à escala?	Configuração de hoje
k-NN	vota a maioria dos k vizinhos mais próximos (por distância)	irregular, com "dentes"	sim (usa distância)	<code>k=5</code>
Regressão logística	combinação linear passada por uma sigmoide, que vira probabilidade	quase uma reta	pouco (mas afeta a otimização)	<code>max_iter=1000</code>
Árvore de decisão	perguntas do tipo "atributo > valor?"	escadinha de retângulos	não	<code>max_depth=4</code>
Naive Bayes	teorema de Bayes supondo atributos independentes dado a classe	curva suave	não	gaussiano
SVM	fronteira com a maior margem possível	reta ou curva flexível (kernel)	sim (usa distância)	<code>kernel="rbf"</code>

Repare que os valores dos hiperparâmetros ($k=5$, $\text{max_depth}=4$) são **escolhas iniciais razoáveis**, não valores otimizados. Como escolher hiperparâmetros com honestidade é tema da próxima sessão.

5. Lendo os resultados

5.1. Acurácia, precisão, recall e F1

Com o mesmo treino e o mesmo teste (135 exemplos), os cinco modelos e o baseline produzem:

Modelo	Acurácia	Precisão	Recall	F1
k-NN ($k=5$)	0,837	0,821	0,846	0,833
Regressão logística	0,859	0,848	0,862	0,855
Árvore de decisão ($\text{max_depth}=4$)	0,815	0,823	0,785	0,803
Naive Bayes gaussiano	0,800	0,771	0,831	0,800
SVM ($\text{kernel}=\text{"rbf"}$)	0,859	0,838	0,877	0,857
Baseline (classe mais comum)	0,519	—	0,000	0,000

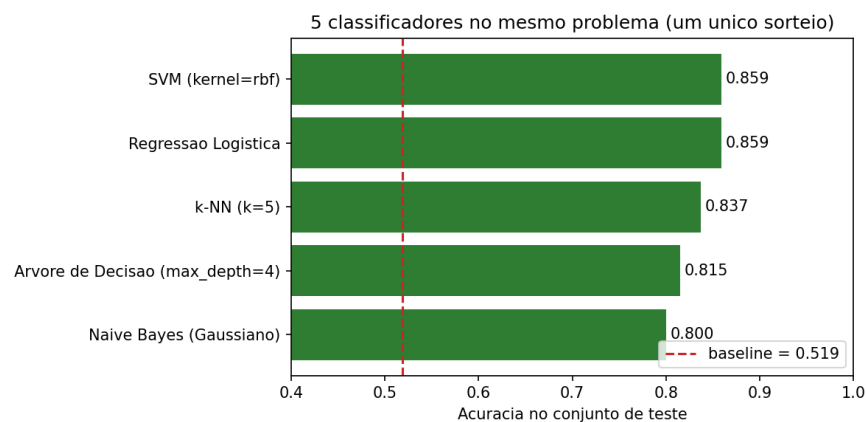


Figura 1 — acurácia no conjunto de teste; a linha tracejada é o baseline.

Todos os modelos superam o baseline com folga: há sinal nos dados. Mas as diferenças **entre** os modelos são de poucos pontos percentuais, e o conjunto de teste tem só 135 exemplos — cada exemplo vale cerca de 0,7 ponto percentual. Uma diferença de 1 ou 2 pontos pode ser só sorte do sorteio.

5.2. Matriz de confusão

A matriz de confusão mostra **como** cada modelo erra. Tomamos a classe 1 (aprovado) como positiva:

- **Falso positivo (FP)**: o modelo aprovou, mas o cliente não deveria ter sido aprovado (aprovação indevida).
- **Falso negativo (FN)**: o modelo negou, mas o cliente era um bom pagador (negação indevida).

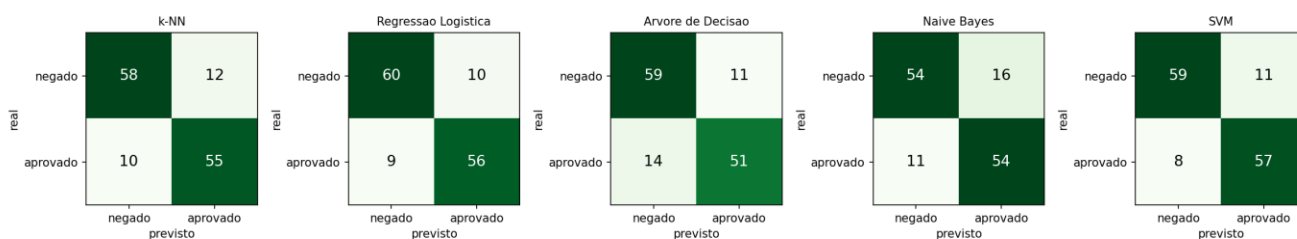


Figura 2 — matrizes de confusão dos cinco modelos no mesmo conjunto de teste (linhas = classe real, colunas = classe prevista).

$\text{precisão} = \text{VP} / (\text{VP} + \text{FP})$ dos que o modelo APROVOU, quantos eram bons pagadores
 $\text{recall} = \text{VP} / (\text{VP} + \text{FN})$ dos que ERAM bons pagadores, quantos o modelo aprovou
 $\text{F1} = 2 * \text{precisão} * \text{recall} / (\text{precisão} + \text{recall})$ média harmônica das duas

Dois modelos de acurácia parecida podem errar de jeitos bem diferentes: o Naive Bayes, por exemplo, cometeu 16 aprovações indevidas contra 10 da regressão logística. Em crédito, os dois erros têm **custos diferentes** — aprovar quem não vai pagar costuma custar mais que negar quem pagaria. Por isso, quem escolhe o modelo precisa perguntar *qual erro importa mais* antes de olhar apenas a acurácia.

6. O papel da escala dos atributos

Repetimos o mesmo experimento **sem** padronizar os atributos numéricos (a imputação e o one-hot continuam):

Modelo	Com escala	Sem escala	Diferença
k-NN (k=5)	0,837	0,748	−0,089
Regressão logística	0,859	0,859	0,000
Árvore de decisão	0,815	0,815	0,000
Naive Bayes	0,800	0,815	+0,015
SVM (rbf)	0,859	0,607	−0,252

k-NN e SVM dependem de **distância**: sem padronizar, a **renda** (na casa dos milhares) domina a **idade** (na casa das dezenas) e o desempenho despenca — o SVM cai de 0,859 para 0,607, bem mais perto do baseline (0,519). A **árvore** compara cada atributo com um limiar, que acompanha a escala, por isso não muda. O **Naive Bayes** modela cada atributo separadamente. A **regressão logística** manteve a acurácia, mas com atributos sem escala a otimização fica mais lenta e os coeficientes deixam de ser comparáveis entre si.

Regra prática: padronize sempre que o modelo usar distância (k-NN, SVM); nos demais, padronizar não machuca e ajuda a interpretar.

7. Um hiperparâmetro em ação: simples demais × complexo demais

Cada algoritmo tem "botões" que controlam sua complexidade. No k-NN é o **k**; na árvore, a profundidade máxima. Veja o que acontece com a acurácia de **treino** e de **teste**:

k (k-NN)	Treino	Teste		max_depth	Treino / Teste
1	1,000	0,719		1	0,832 / 0,822
5	0,841	0,837		4	0,876 / 0,815
15	0,851	0,852		8	0,978 / 0,793
71	0,838	0,881		sem limite	1,000 / 0,800

Com $k=1$, o k-NN acerta 100% do treino (cada ponto é o próprio vizinho), mas cai no teste. A árvore sem limite de profundidade também decora o treino. É o mesmo padrão da regressão polinomial de grau alto: um modelo flexível demais **memoriza o ruído**.

Repare também que $k=71$ teve a *melhor* acurácia de teste. Seria tentador escolhê-lo por isso — mas escolher k olhando para o teste é trapaça: o teste deixa de ser "dados nunca visto". Como ajustar hiperparâmetros sem trair o teste é o tema da sessão seguinte.

8. O problema de um único sorteio

Até aqui, todos os resultados vêm de **uma** divisão treino/teste. Repetimos o experimento inteiro com **10 divisões diferentes** (só mudando `random_state`):

Modelo	Média	Desvio	Mínimo	Máximo
k-NN (k=5)	0,801	0,012	0,785	0,822
Regressão logística	0,835	0,026	0,785	0,867
Árvore de decisão	0,830	0,026	0,793	0,889
Naive Bayes	0,799	0,029	0,756	0,837
SVM (rbf)	0,841	0,019	0,807	0,867

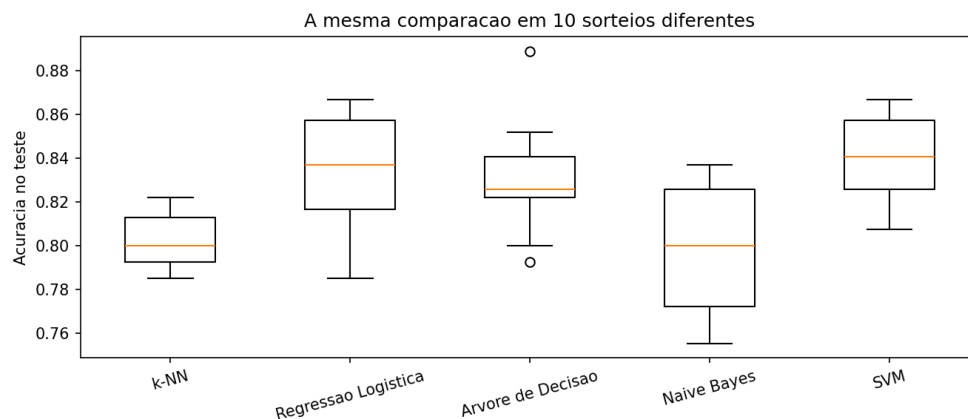


Figura 3 — distribuição da acurácia de cada modelo em 10 divisões treino/teste diferentes.

O primeiro colocado mudou de sorteio para sorteio: a regressão logística venceu 5 vezes, o SVM 3 vezes e a árvore de decisão 2 vezes. Além disso, as faixas de desempenho **se sobrepõem** bastante. A conclusão é importante: **uma única divisão treino/teste é uma estimativa ruidosa**, e declarar um "vencedor" com base nela é arriscado.

Um cuidado importante

Nenhum dos cinco algoritmos é "o melhor" de forma absoluta. Neste problema, regressão logística e SVM ficaram no grupo da frente, mas a diferença para os demais é pequena perto da variação entre sorteios. A resposta responsável a "qual é o melhor modelo?" é: **depende de como você mede, e é preciso medir de um jeito que mostre a incerteza**. Isso leva à validação cruzada, às métricas adequadas ao problema e às curvas de aprendizado — o conteúdo da próxima sessão.

O que guardar desta sessão

- Uma comparação justa muda **só o algoritmo**: mesmos dados, mesma divisão, mesmo pré-processamento e mesmas métricas.
- O **baseline** (sempre a classe mais comum) é o piso; um modelo só vale se o supera com folga.

- Coloque o pré-processamento **dentro do Pipeline**: as estatísticas são aprendidas só com o treino.
- Acurácia isolada esconde **como** o modelo erra. Use matriz de confusão, precisão, recall e F1, e pergunte qual erro custa mais.
- **k-NN e SVM exigem padronização** (usam distância); árvore, Naive Bayes e regressão logística praticamente não dependem dela.
- Modelos flexíveis demais decoram o treino (sobreajuste); escolher hiperparâmetros olhando o teste é trapaça.
- Uma única divisão treino/teste é ruidosa: o "melhor modelo" muda com o sorteio.

Arquivos desta sessão

- [lab08_parte1_demo.py](#) — demonstração guiada (7 blocos), gera as três figuras desta apostila.
- [lab08_parte1_comparacao_classificadores.ipynb](#) — a mesma demonstração em notebook, com explicações entre as células.
- [lab08_parte1_exercicios.py](#) — mini-laboratório do aluno (preencha os **# TODO** e responda o RELATORIO).

Para executar: `python lab08_parte1_demo.py` (é necessário ter `numpy`, `pandas`, `matplotlib` e `scikit-learn` instalados).